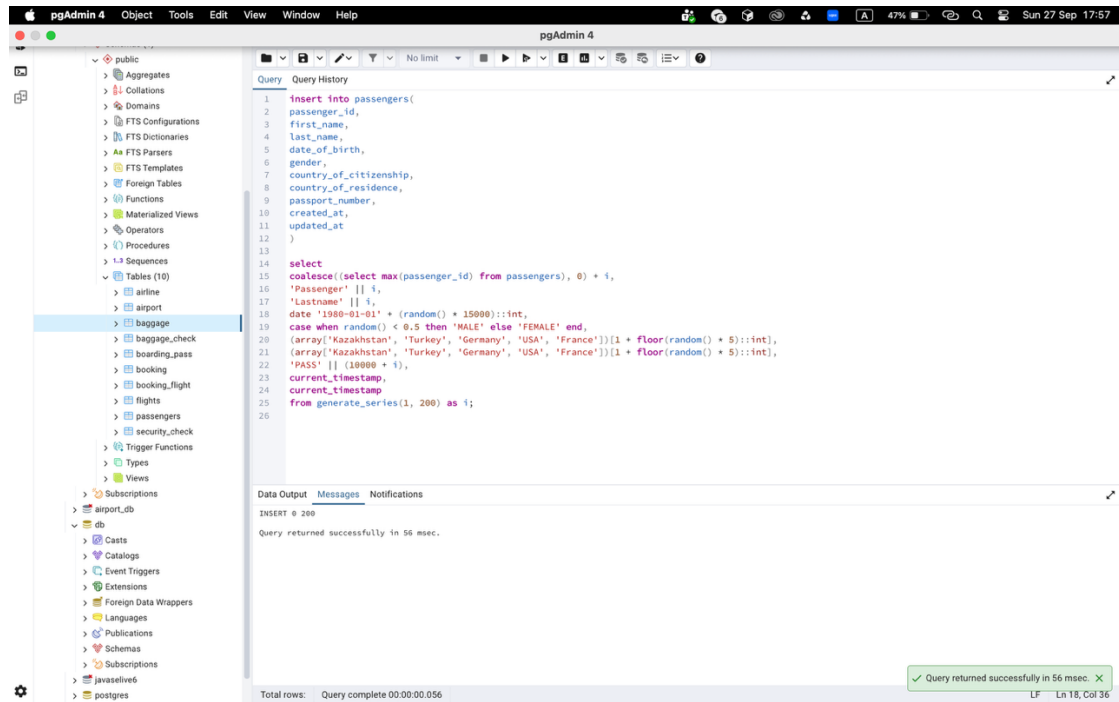


3-LAB

DML

Task-1. Generate and Insert 200 random rows

SQL-Code:

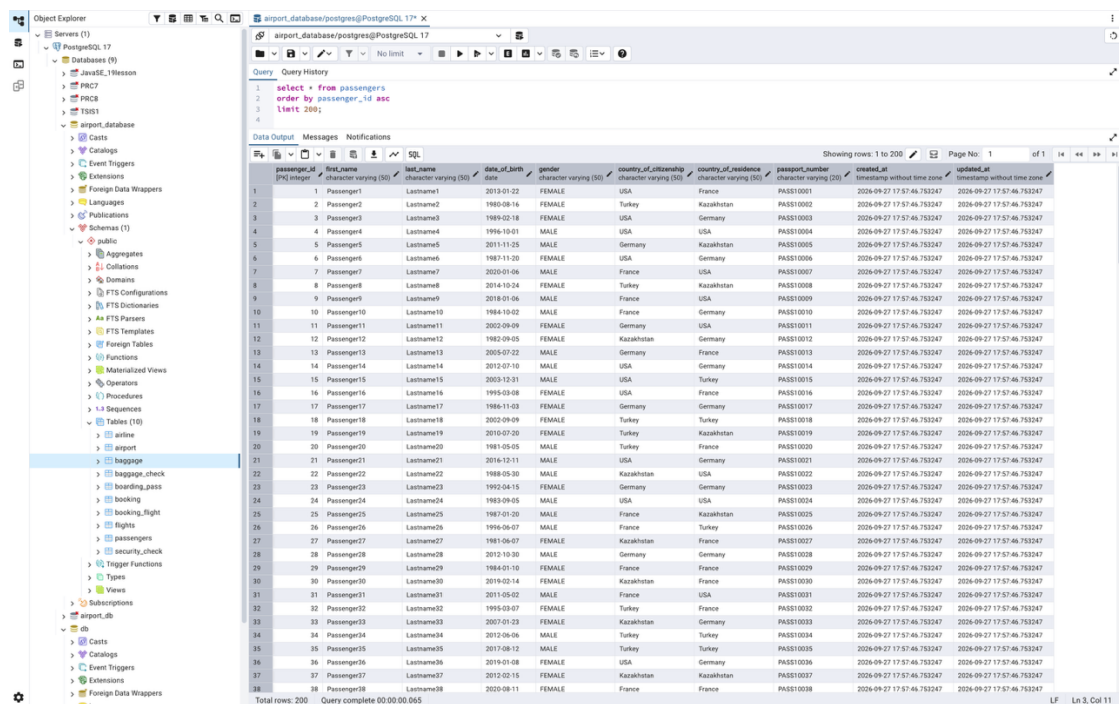


The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane displays the database structure, including 'public', 'airline', 'airport', 'baggage', 'booking', 'flights', 'passengers', 'security_check', 'Types', 'Views', 'Subscriptions', 'airport_db', 'db', 'Casts', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas', 'Subscriptions', 'javaload', and 'postgres'. The main pane shows a SQL query being executed. The query is as follows:

```
1 insert into passengers(  
2 passenger_id,  
3 first_name,  
4 last_name,  
5 date_of_birth,  
6 gender,  
7 country_of_citizenship,  
8 country_of_residence,  
9 passport_number,  
10 created_at,  
11 updated_at  
12 )  
13  
14  
15 select  
16 coalesce((select max(passenger_id) from passengers), 0) + i,  
17 'Passenger' || i,  
18 'Lastname' || i,  
19 date '1980-01-01' + (random() * 15000)::int,  
20 case when random() < 0.5 then 'MALE' else 'FEMALE' end,  
21 (array['Kazakhstan', 'Turkey', 'Germany', 'USA', 'France'])[1 + floor(random() * 5)::int],  
22 (array['Kazakhstan', 'Turkey', 'Germany', 'USA', 'France'])[1 + floor(random() * 5)::int],  
23 'PASS' || (10000 + i),  
24 current_timestamp,  
25 from generate_series(1, 200) as i;  
26
```

The 'Data Output' pane shows the result of the query: 'INSERT 0 200'. Below this, a message states: 'Query returned successfully in 56 msec.' At the bottom, a status bar indicates 'Total rows: Query complete 00:00:00.056' and 'Ln 18, Col 36'.

Result after inserting data:



The screenshot shows the pgAdmin 4 interface with the 'airline_database/postgres/PostgreSQL 17' database selected. The 'Query History' pane shows the same SQL query as in the previous screenshot. The 'Data Output' pane displays the result of the query, showing 200 rows of data. The data is presented in a table with the following columns: passenger_id, first_name, last_name, date_of_birth, gender, country_of_citizenship, country_of_residence, passport_number, created_at, and updated_at. The table shows the first 20 rows of data, with the last row being '200 Passenger20 Lastname20 2020-09-11 FEMALE France France PASS10020 2024-09-27 17:57:46.753247 2024-09-27 17:57:46.753247'. The status bar at the bottom indicates 'Total rows: 200 Query complete 00:00:00.065' and 'Ln 3, Col 11'.

Task-2. Adding row into Airline table

SQL-Code:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane displays the database structure for 'airport_database', with the 'airline' table selected under the 'Tables' folder. The main pane shows a SQL query window with the following code:

```
1 INSERT INTO airline (
2     airline_id,
3     airline_code,
4     airline_name,
5     airline_country,
6     created_at,
7     updated_at
8 )
9 VALUES (
10     (SELECT COALESCE(MAX(airline_id), 0) + 1 FROM airline),
11     'KAZ',
12     'KazAir',
13     'Kazakhstan',
14     CURRENT_TIMESTAMP,
15     CURRENT_TIMESTAMP
16 );
17
```

Below the query window, the 'Data Output' pane shows the result of the query: 'INSERT 0 1'. A message below indicates 'Query returned successfully in 132 msec.' The status bar at the bottom shows 'Total rows: Query complete 00:00:00.132'.

Result after adding row:

The screenshot shows the same SQL Server Enterprise Manager interface. The 'airline' table is still selected in the 'Object Explorer'. The main pane shows a SQL query window with the following code:

```
1 select * from airline
2 where airline_name = 'KazAir';
3
```

Below the query window, the 'Data Output' pane shows the result of the query. The table has the following columns: 'airline_id' (integer), 'airline_code' (character varying (30)), 'airline_name' (character varying (50)), 'airline_country' (character varying (50)), 'created_at' (timestamp without time zone), 'updated_at' (timestamp without time zone), and 'info' (character varying (50)). The result shows one row with the following values:

airline_id	airline_code	airline_name	airline_country	created_at	updated_at	info
1	KAZ	KazAir	Kazakhstan	2026-09-27 19:05:20.781285	2026-09-27 19:05:20.781285	[null]

The status bar at the bottom shows 'Total rows: 1 Query complete 00:00:00.077'. A green message box at the bottom right indicates 'Successfully run. Total query runtime: 77 msec. 1 rows affected.'

Task-3. Update airline country

SQL-Code:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the database structure, with the 'airline' table selected under the 'airline_database' schema. The central pane shows a query window with the following SQL code:

```
1 update Airline
2 set airline_country = 'Turkey'
3 where airline_name = 'KazAir';
```

The bottom pane shows the 'Data Output' tab with the following message:

```
UPDATE 1
Query returned successfully in 101 msec.
```

A status bar at the bottom indicates 'Total rows: Query complete 00:00:00.101' and 'Query returned successfully in 101 msec.'.

Result after updating:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the database structure, with the 'airline' table selected under the 'airline_database' schema. The central pane shows a query window with the following SQL code:

```
1 select * from Airline
2 where airline_name = 'KazAir';
```

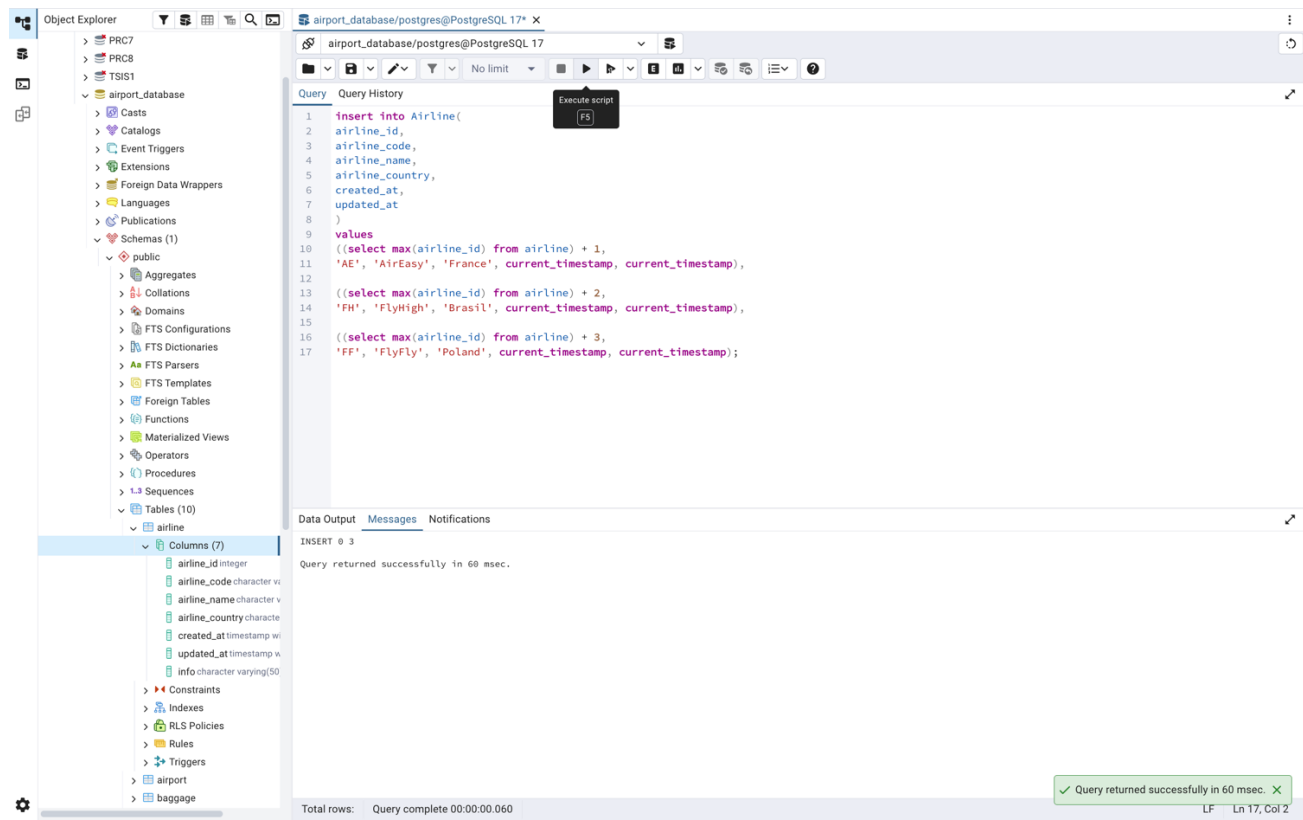
The bottom pane shows the 'Data Output' tab with the following table:

airline_id	airline_code	airline_name	airline_country	created_at	updated_at	info
[PK] integer	character varying (30)	character varying (50)	character varying (50)	timestamp without time zone	timestamp without time zone	info character varying (50)
1	KAZ	KazAir	Turkey	2026-09-27 19:05:20.781285	2026-09-27 19:05:20.781285	[null]

A status bar at the bottom indicates 'Total rows: 1 Query complete 00:00:00.077' and 'Successfully run. Total query runtime: 77 msec. 1 rows affected.'.

Task-4. Add three airlines at once

SQL-Code:

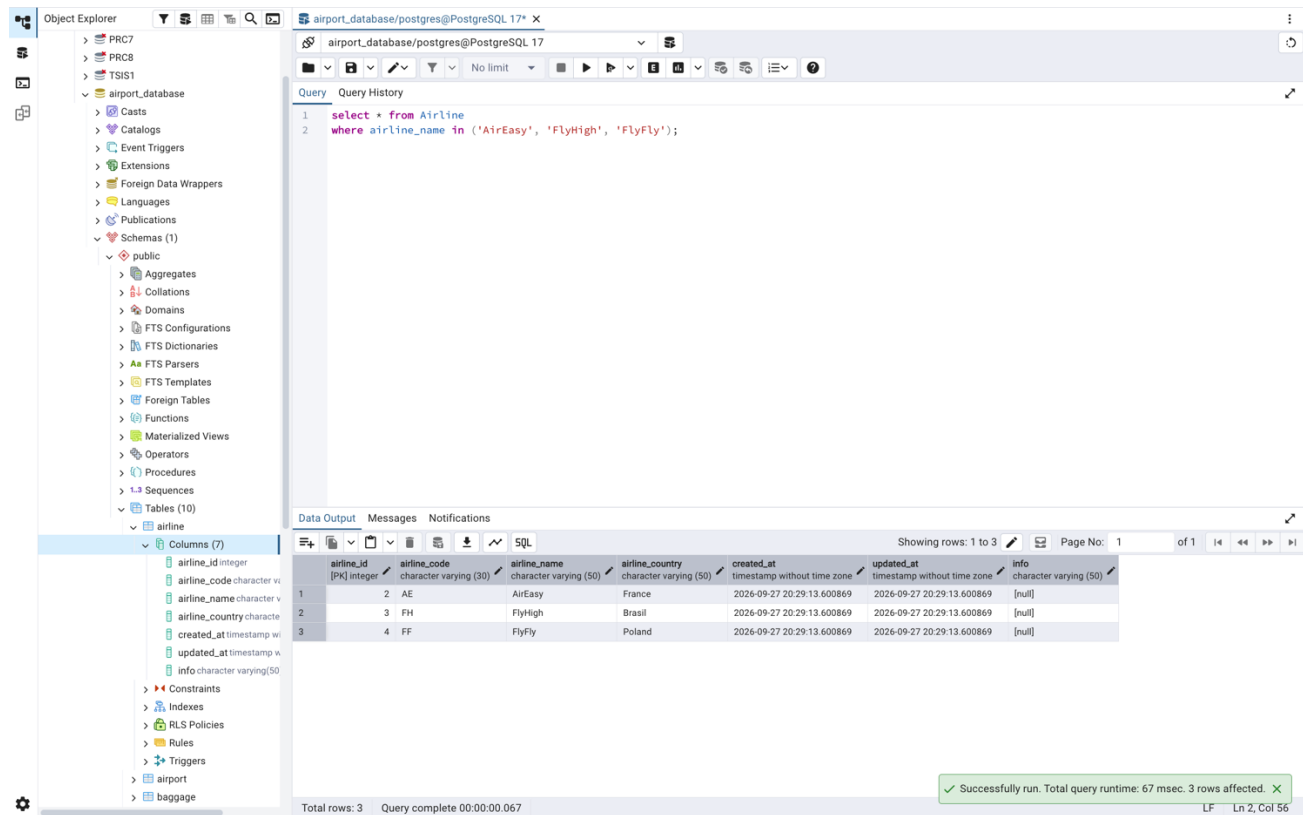


The screenshot shows the PostgreSQL IDE interface. On the left, the Object Explorer displays the database structure, including the 'airline' table. The main query editor contains an INSERT statement that adds three new airlines: 'AirEasy' (France), 'FlyHigh' (Brasil), and 'FlyFly' (Poland). The Data Output pane shows the successful execution of the query, returning 3 rows in 60 milliseconds.

```
1 insert into Airline(
2   airline_id,
3   airline_code,
4   airline_name,
5   airline_country,
6   created_at,
7   updated_at
8 )
9 values
10 ((select max(airline_id) from airline) + 1,
11  'AE', 'AirEasy', 'France', current_timestamp, current_timestamp),
12
13 ((select max(airline_id) from airline) + 2,
14  'FH', 'FlyHigh', 'Brasil', current_timestamp, current_timestamp),
15
16 ((select max(airline_id) from airline) + 3,
17  'FF', 'FlyFly', 'Poland', current_timestamp, current_timestamp);
```

Query returned successfully in 60 msec.

Result after adding:



The screenshot shows the PostgreSQL IDE interface. The main query editor contains a SELECT statement that retrieves all records from the 'Airline' table where the 'airline_name' is 'AirEasy', 'FlyHigh', or 'FlyFly'. The Data Output pane displays the results of the query, showing 3 rows with columns: airline_id, airline_code, airline_name, airline_country, created_at, updated_at, and info.

```
1 select * from Airline
2 where airline_name in ('AirEasy', 'FlyHigh', 'FlyFly');
```

airline_id	airline_code	airline_name	airline_country	created_at	updated_at	info
1	AE	AirEasy	France	2026-09-27 20:29:13.600869	2026-09-27 20:29:13.600869	[null]
2	FH	FlyHigh	Brasil	2026-09-27 20:29:13.600869	2026-09-27 20:29:13.600869	[null]
3	FF	FlyFly	Poland	2026-09-27 20:29:13.600869	2026-09-27 20:29:13.600869	[null]

Successfully run. Total query runtime: 67 msec. 3 rows affected.

Task-5. Delete all flights whose arrival in 2024

SQL-Code:

The screenshot shows the PostgreSQL IDE interface. On the left, the Object Explorer displays the database structure, including the 'airline' table. The main query editor contains the following SQL code:

```
1 delete from Flights
2 where sch_arrival_time >= '2024-01-01' and sch_arrival_time < '2025-01-01';
```

The Data Output tab shows the result of the query: "DELETE 73" and "Query returned successfully in 57 msec." A green status bar at the bottom right confirms: "Query returned successfully in 57 msec. X". The status bar also indicates "Total rows: Query complete 00:00:00.057" and "LF Ln 2, Col 62".

Result after deleting:

The screenshot shows the PostgreSQL IDE interface after the deletion query. The main query editor contains the following SQL code:

```
1 select * from flights;
```

The Data Output tab displays the result of the query, showing 127 rows. The status bar at the bottom indicates "Total rows: 127" and "Query complete 00:00:00.067". The status bar also shows "LF Ln 1, Col 23".

flightId	sch_departure_time	sch_arrival_time	departing_airportId	arriving_airportId	departing_gate	arriving_gate	airline_id	act_departure_time	act_arrival_time
1	2025-06-19 08:54:48.277841	2025-06-19 12:54:48.277841	40	64	A6	B13	4	2025-06-19 09:04:48.277841	2025
2	2023-07-24 14:32:04.056808	2023-07-24 18:32:04.056808	113	127	A6	B10	2	2023-07-24 14:42:04.056808	2023
3	2023-06-11 20:29:17.429703	2023-06-12 00:29:17.429703	149	43	A1	B5	4	2023-06-11 20:39:17.429703	2023
4	2025-02-13 23:49:00.847907	2025-02-14 03:49:00.847907	31	128	A2	B4	4	2025-02-13 23:59:00.847907	2025
5	2023-01-12 17:18:20.29008	2023-01-12 21:18:20.29008	100	133	A4	B13	1	2023-01-12 17:28:20.29008	2023
6	2025-10-12 17:39:51.763407	2025-10-12 21:39:51.763407	190	17	A9	B10	2	2025-10-12 17:49:51.763407	2025
7	2025-05-09 22:40:10.193973	2025-05-10 02:40:10.193973	71	63	A13	B16	2	2025-05-09 22:50:10.193973	2025
8	2023-03-26 23:21:11.535899	2023-03-27 03:21:11.535899	76	68	A17	B12	1	2023-03-26 23:31:11.535899	2023
9	2025-10-30 14:40:20.088929	2025-10-30 18:40:20.088929	58	47	A2	B4	4	2025-10-30 14:50:20.088929	2025
10	2023-08-30 04:23:52.171567	2023-08-30 08:23:52.171567	113	165	A2	B14	3	2023-08-30 04:33:52.171567	2023
11	2023-10-27 21:57:26.599214	2023-10-28 01:57:26.599214	133	191	A4	B15	1	2023-10-27 22:07:26.599214	2023
12	2025-02-24 15:43:55.381869	2025-02-24 19:43:55.381869	27	112	A7	B1	1	2025-02-24 15:53:55.381869	2025
13	2023-03-25 20:40:45.750765	2023-03-26 00:40:45.750765	10	51	A7	B11	1	2023-03-25 20:50:45.750765	2023
14	2025-12-05 11:39:02.350647	2025-12-05 15:39:02.350647	124	83	A20	B1	3	2025-12-05 11:49:02.350647	2025
15	2023-10-05 00:22:42.285849	2023-10-05 04:22:42.285849	3	151	A9	B9	3	2023-10-05 00:32:42.285849	2023
16	2025-12-19 05:47:39.980446	2025-12-19 09:47:39.980446	160	151	A19	B8	2	2025-12-19 05:57:39.980446	2025
17	2023-03-12 22:32:27.917148	2023-03-13 02:32:27.917148	100	113	A6	B8	3	2023-03-12 22:42:27.917148	2023
18	2025-06-06 06:30:19.862562	2025-06-06 10:30:19.862562	112	112	A10	B16	3	2025-06-06 06:40:19.862562	2025
19	2023-11-22 20:05:09.905297	2023-11-23 00:05:09.905297	116	117	A5	B14	3	2023-11-22 20:15:09.905297	2023
20	2025-05-16 16:28:40.456002	2025-05-16 20:28:40.456002	134	68	A19	B7	3	2025-05-16 16:38:40.456002	2025
21	2025-11-29 08:43:22.022507	2025-11-29 12:43:22.022507	80	57	A13	B8	4	2025-11-29 08:53:22.022507	2025
22	2025-10-11 13:50:07.683503	2025-10-11 17:50:07.683503	138	33	A10	B19	4	2025-10-11 14:00:07.683503	2025
23	2025-02-16 08:47:54.341652	2025-02-16 12:47:54.341652	114	135	A11	B19	3	2025-02-16 08:57:54.341652	2025
24	2023-03-20 19:20:25.015818	2023-03-20 23:20:25.015818	111	67	A11	B11	3	2023-03-20 19:30:25.015818	2023
25	2025-11-08 03:24:28.123539	2025-11-08 07:24:28.123539	91	176	A20	B2	2	2025-11-08 03:34:28.123539	2025
26	2025-05-03 06:16:16.109095	2025-05-03 10:16:16.109095	41	85	A8	B6	4	2025-05-03 06:26:16.109095	2025
27	2025-05-04 21:24:20.888713	2025-05-05 01:24:20.888713	185	133	A18	B19	1	2025-05-04 21:34:20.888713	2025
28	2023-09-30 15:39:18.249756	2023-09-30 19:39:18.249756	161	68	A12	B14	3	2023-09-30 15:49:18.249756	2023
29	2025-02-09 03:46:17.011711	2025-02-09 07:46:17.011711	80	73	A5	B12	4	2025-02-09 03:56:17.011711	2025

Task-6. Increase price of data in booking table

SQL-Code:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the database structure, with the 'booking' table selected under the 'airline' schema. The main pane shows the execution of the following SQL query:

```
1 update booking
2 set ticket_price = ticket_price * 1.15;
```

The Data Output tab shows the results of the query:

UPDATE 200
Query returned successfully in 86 msec.

Total rows: Query complete 00:00:00.086

Result after increasing price:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the database structure, with the 'booking' table selected under the 'airline' schema. The main pane shows the execution of the following SQL query:

```
1 select booking_id, ticket_price
2 from booking;
```

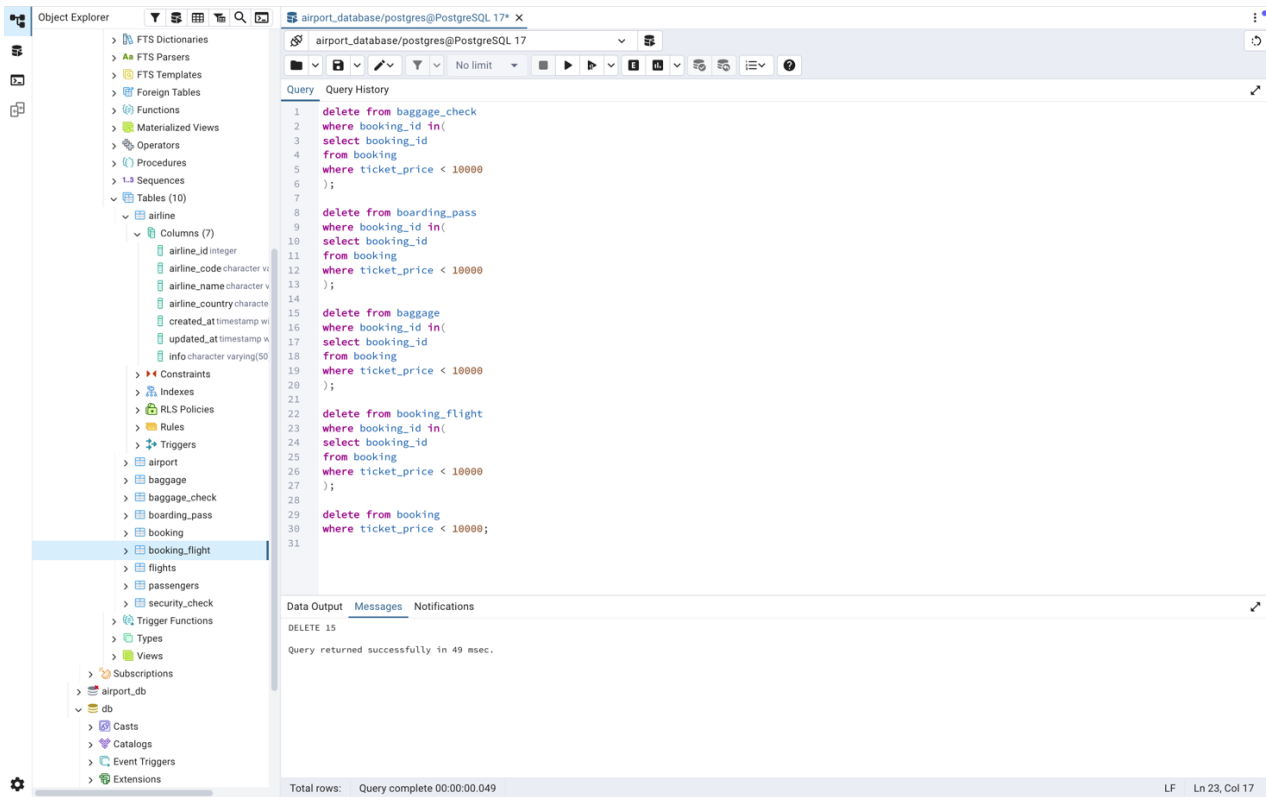
The Data Output tab shows the results of the query, displaying 200 rows. The first 20 rows are shown in the table below:

booking_id [PK] integer	ticket_price numeric (7,2)
1	37427.90
2	15560.65
3	8276.55
4	47201.75
5	54013.20
6	36560.80
7	11491.95
8	46368.00
9	38663.00
10	43277.95
11	48938.25
12	23990.15
13	23307.05
14	25060.80
15	30961.45
16	36026.05
17	14703.90
18	12859.30
19	52725.20
20	43777.05
21	29729.80
22	36622.90
23	31053.45
24	41854.25
25	9286.25
26	26947.95
27	17316.70
28	30089.75
29	16764.70

Total rows: 200 Query complete 00:00:00.070

Task-7. Delete all tickets where price less than 10000

SQL-Code:

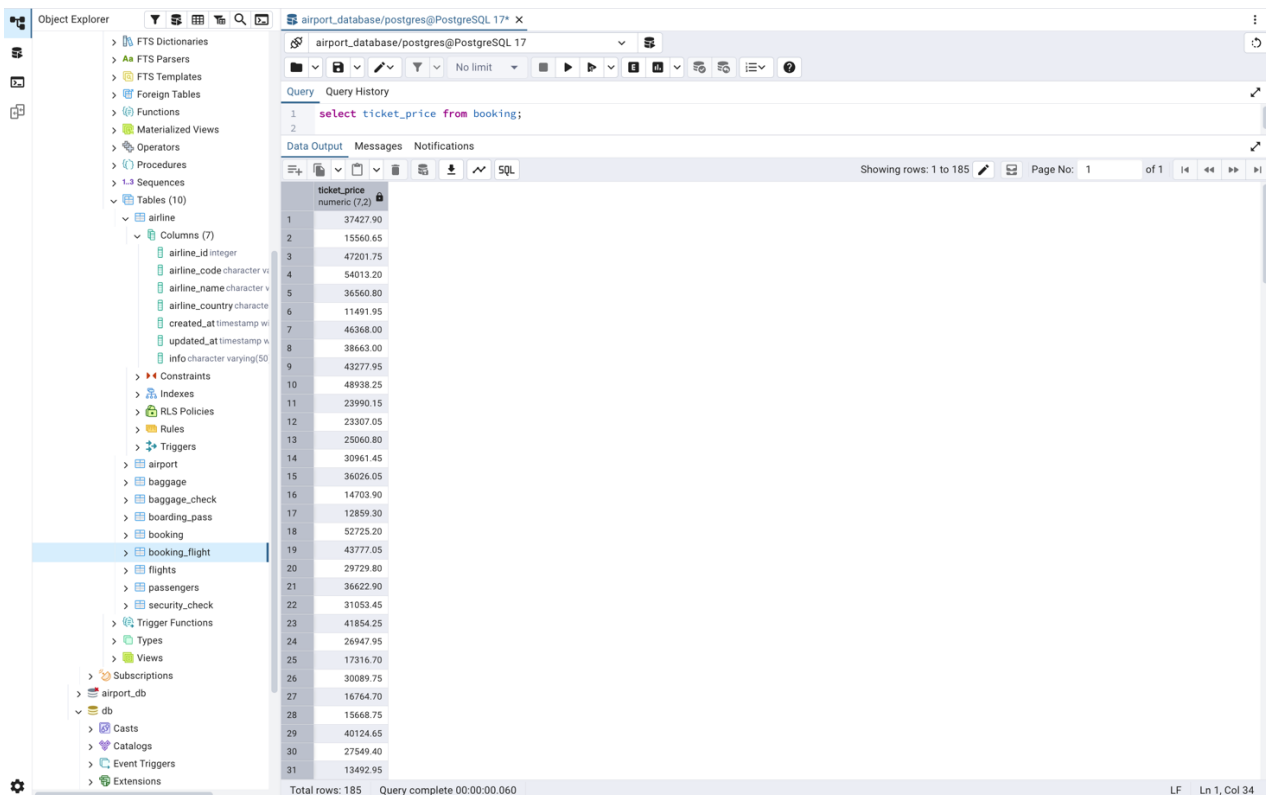


The screenshot shows the PostgreSQL IDE interface. The Object Explorer on the left displays the database structure, including the 'booking_flight' table. The main query editor contains the following SQL code:

```
1 delete from baggage_check
2 where booking_id in(
3 select booking_id
4 from booking
5 where ticket_price < 10000
6 );
7
8 delete from boarding_pass
9 where booking_id in(
10 select booking_id
11 from booking
12 where ticket_price < 10000
13 );
14
15 delete from baggage
16 where booking_id in(
17 select booking_id
18 from booking
19 where ticket_price < 10000
20 );
21
22 delete from booking_flight
23 where booking_id in(
24 select booking_id
25 from booking
26 where ticket_price < 10000
27 );
28
29 delete from booking
30 where ticket_price < 10000;
31
```

The Data Output tab shows the result of the first query: DELETE 15. The status bar indicates 'Query returned successfully in 49 msec.' and 'Total rows: Query complete 00:00:00.049'.

Result after deleting:



The screenshot shows the PostgreSQL IDE interface after the deletion query. The main query editor contains the following SQL code:

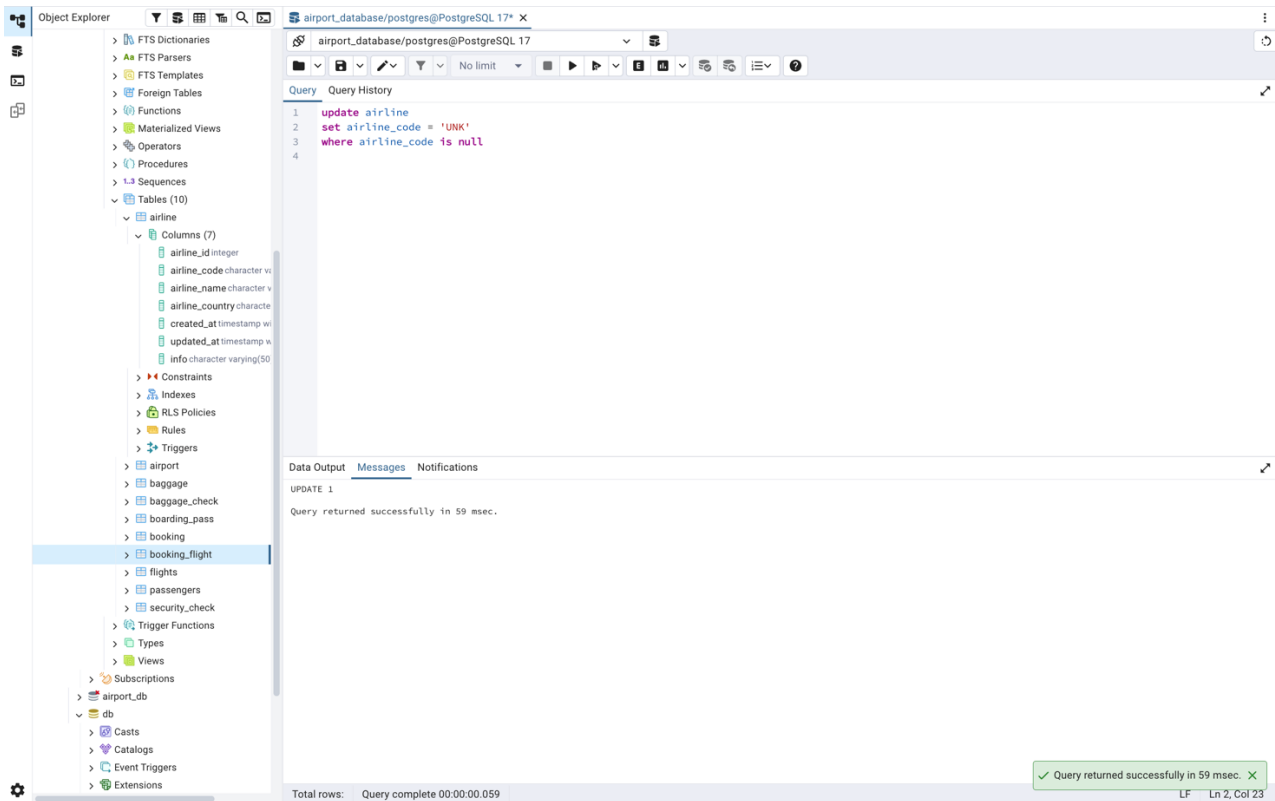
```
1 select ticket_price from booking;
2
```

The Data Output tab shows the result of the query, displaying a list of ticket prices. The status bar indicates 'Showing rows: 1 to 185' and 'Page No: 1 of 1'. The status bar also shows 'Total rows: 185' and 'Query complete 00:00:00.060'.

ticket_price
37427.90
15560.65
47201.75
54013.20
36560.80
11491.95
46368.00
38663.00
43277.95
48938.25
23990.15
23307.05
25060.80
30961.45
36026.05
14703.90
12859.30
52725.20
43777.05
29729.80
36622.90
31053.45
41854.25
26947.95
17316.70
30089.75
16764.70
15668.75
40124.65
27549.40
13492.95

Task-8. Set a default code where airline_code missing

SQL-code:



The screenshot shows the PostgreSQL IDE interface. On the left, the Object Explorer displays the database structure, including the 'airline' table with columns: airline_id, airline_code, airline_name, airline_country, created_at, updated_at, and info. The main query editor contains the following SQL code:

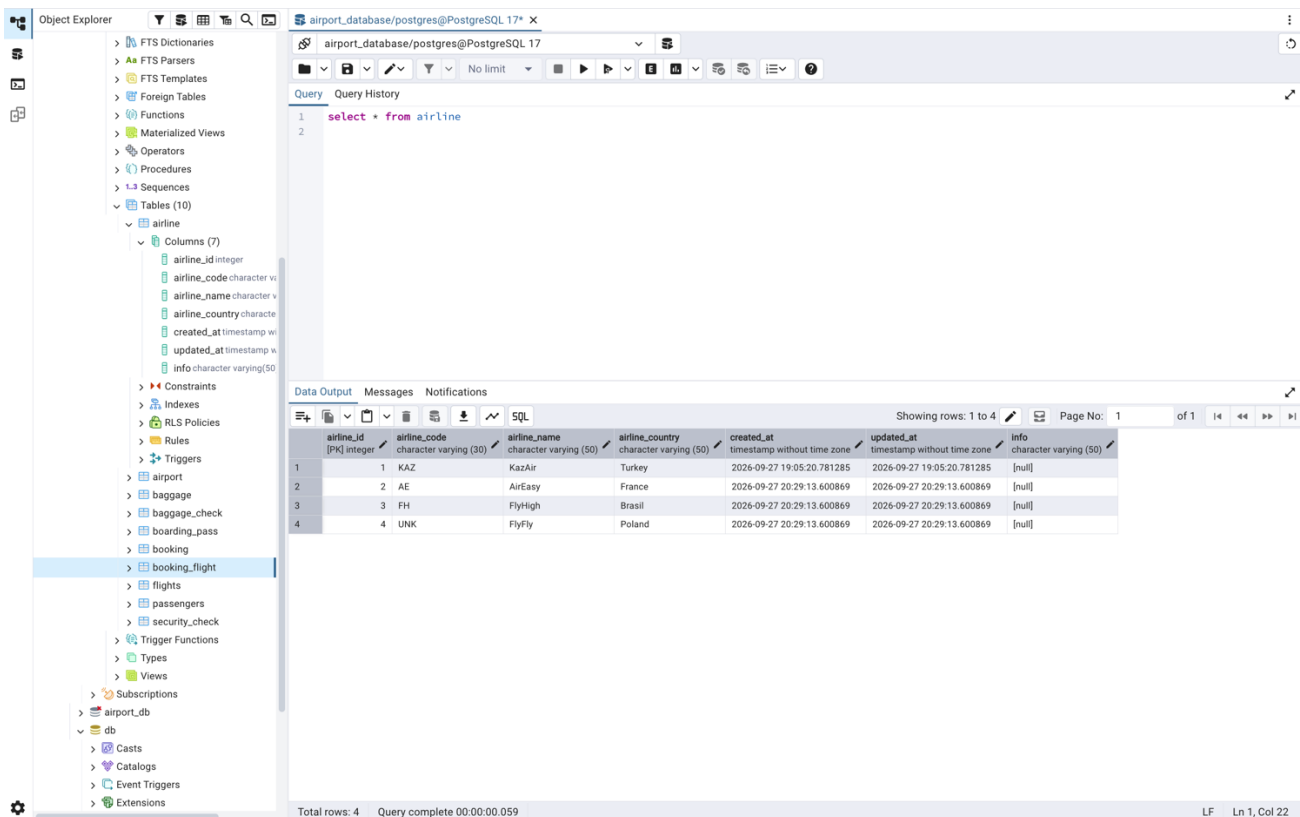
```
1 update airline
2 set airline_code = 'UNK'
3 where airline_code is null
4
```

The Data Output pane shows the result of the query:

```
UPDATE 1
Query returned successfully in 59 msec.
```

A status message at the bottom right indicates: "Query returned successfully in 59 msec." The status bar at the bottom shows "Total rows: Query complete 00:00:00.059" and "LF Ln 2, Col 23".

Result after updating:



The screenshot shows the PostgreSQL IDE interface after the update. The main query editor contains the following SQL code:

```
1 select * from airline
2
```

The Data Output pane displays the result of the query as a table with 7 columns: airline_id, airline_code, airline_name, airline_country, created_at, updated_at, and info. The table contains 4 rows of data:

airline_id	airline_code	airline_name	airline_country	created_at	updated_at	info
1	KAZ	KazAir	Turkey	2026-09-27 19:05:20.781285	2026-09-27 19:05:20.781285	[null]
2	AE	AirEasy	France	2026-09-27 20:29:13.600869	2026-09-27 20:29:13.600869	[null]
3	FH	FlyHigh	Brasil	2026-09-27 20:29:13.600869	2026-09-27 20:29:13.600869	[null]
4	UNK	FlyFly	Poland	2026-09-27 20:29:13.600869	2026-09-27 20:29:13.600869	[null]

The status bar at the bottom shows "Total rows: 4 Query complete 00:00:00.059" and "LF Ln 1, Col 22".

Task-9. Delete all baggage check records created before June 1, 2023, whose result is still 'Not checked'.

SQL-Code:

The screenshot shows the PostgreSQL IDE interface. On the left, the Object Explorer displays the database schema, including tables like 'airline', 'baggage', and 'baggage_check'. The main query editor contains the following SQL code:

```
1 delete from baggage_check
2 where created_at < '2023-06-01' and check_result = 'Not checked'
3
```

The Data Output pane at the bottom shows the result of the query:

```
DELETE 47
Query returned successfully in 96 msec.
```

A status bar at the bottom indicates: "Query returned successfully in 96 msec." and "Total rows: Query complete 00:00:00.096".

Result after deleting:

The screenshot shows the PostgreSQL IDE interface after the deletion. The main query editor contains the following SQL code:

```
1 select * from baggage_check
2
```

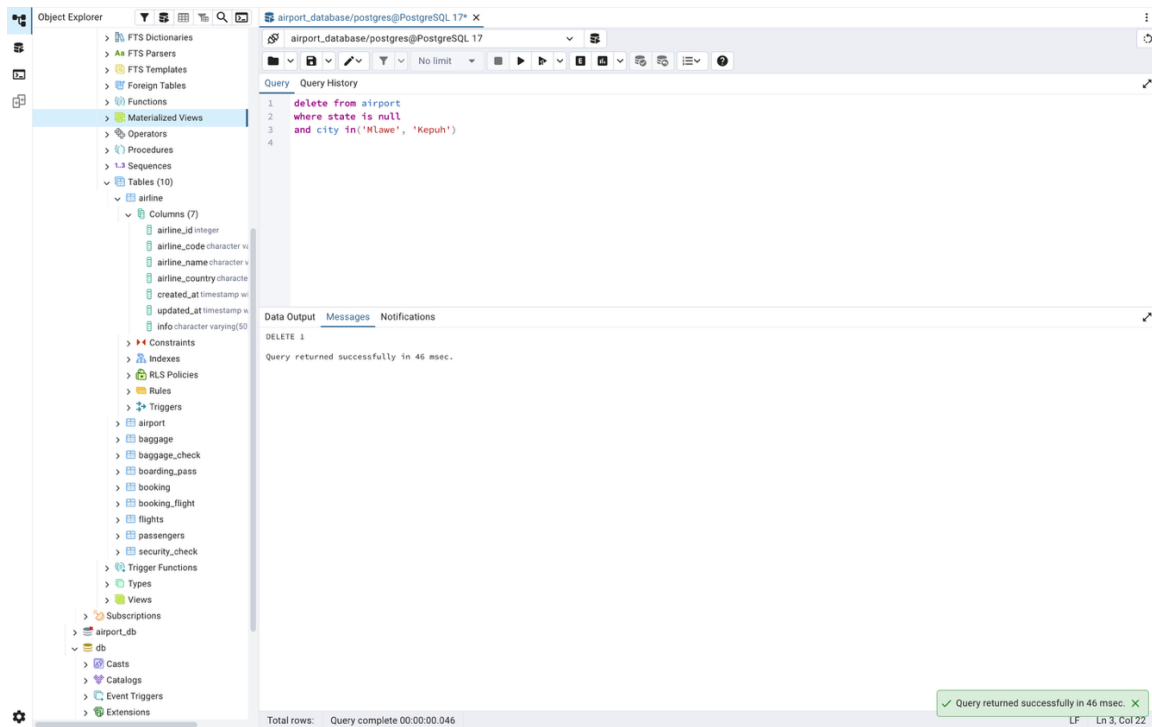
The Data Output pane displays the result of the query, showing 30 rows of data. The columns are: baggage_check_id, check_result, created_at, updated_at, booking_id, and passenger_id. The data is as follows:

baggage_check_id	check_result	created_at	updated_at	booking_id	passenger_id
1	Checked	2022-08-11 20:10:37.715706	2026-09-27 21:15:14.273176	1	35
2	Not checked	2024-05-10 19:53:30.325331	2026-09-27 21:15:14.273176	2	157
3	Checked	2022-10-17 16:33:24.203833	2026-09-27 21:15:14.273176	5	88
4	Checked	2022-12-03 13:14:24.386444	2026-09-27 21:15:14.273176	7	145
5	Checked	2023-07-18 09:27:23.616159	2026-09-27 21:15:14.273176	9	52
6	Not checked	2024-06-13 12:42:45.45081	2026-09-27 21:15:14.273176	10	93
7	Checked	2023-02-20 22:44:13.831038	2026-09-27 21:15:14.273176	11	123
8	Not checked	2023-12-10 11:38:55.222192	2026-09-27 21:15:14.273176	12	193
9	Checked	2023-01-14 16:38:56.422547	2026-09-27 21:15:14.273176	13	60
10	Not checked	2023-09-20 04:58:16.680417	2026-09-27 21:15:14.273176	14	137
11	Checked	2024-04-07 09:46:24.728681	2026-09-27 21:15:14.273176	15	95
12	Not checked	2023-12-03 14:21:02.804284	2026-09-27 21:15:14.273176	16	3
13	Checked	2024-02-14 02:58:45.675129	2026-09-27 21:15:14.273176	17	44
14	Not checked	2024-05-05 12:37:58.175967	2026-09-27 21:15:14.273176	18	109
15	Checked	2023-07-14 14:24:01.987566	2026-09-27 21:15:14.273176	19	63
16	Checked	2024-04-26 21:35:33.08893	2026-09-27 21:15:14.273176	21	82
17	Not checked	2024-06-25 13:17:42.188272	2026-09-27 21:15:14.273176	22	170
18	Checked	2023-05-02 06:50:46.533778	2026-09-27 21:15:14.273176	23	40
19	Not checked	2024-04-07 18:43:44.312132	2026-09-27 21:15:14.273176	24	166
20	Not checked	2024-07-01 15:00:59.885682	2026-09-27 21:15:14.273176	26	121
21	Checked	2022-02-02 19:20:43.422386	2026-09-27 21:15:14.273176	27	69
22	Checked	2023-03-20 13:33:10.523642	2026-09-27 21:15:14.273176	29	146
23	Checked	2024-03-20 07:39:21.566567	2026-09-27 21:15:14.273176	31	118
24	Checked	2023-03-17 16:02:45.324229	2026-09-27 21:15:14.273176	33	142
25	Not checked	2024-09-01 20:06:11.260456	2026-09-27 21:15:14.273176	34	75
26	Checked	2022-09-05 22:35:06.178971	2026-09-27 21:15:14.273176	35	28
27	Checked	2023-01-03 20:10:07.126286	2026-09-27 21:15:14.273176	37	142
28	Checked	2023-09-06 21:37:21.553073	2026-09-27 21:15:14.273176	39	36
29	Checked	2022-03-20 15:03:29.198453	2026-09-27 21:15:14.273176	41	108
30	Not checked	2024-09-13 18:38:52.056433	2026-09-27 21:15:14.273176	42	14

The status bar at the bottom indicates: "Total rows: 138 Query complete 00:00:00.062".

Task-10. Delete all entries from the airport table where state is missing (NULL) and the city is either 'Mlawe' or 'Kepuh'.

SQL-Code:



The screenshot shows the PostgreSQL IDE interface. On the left, the Object Explorer displays the database structure, including the 'airport' table. The main query editor contains the following SQL code:

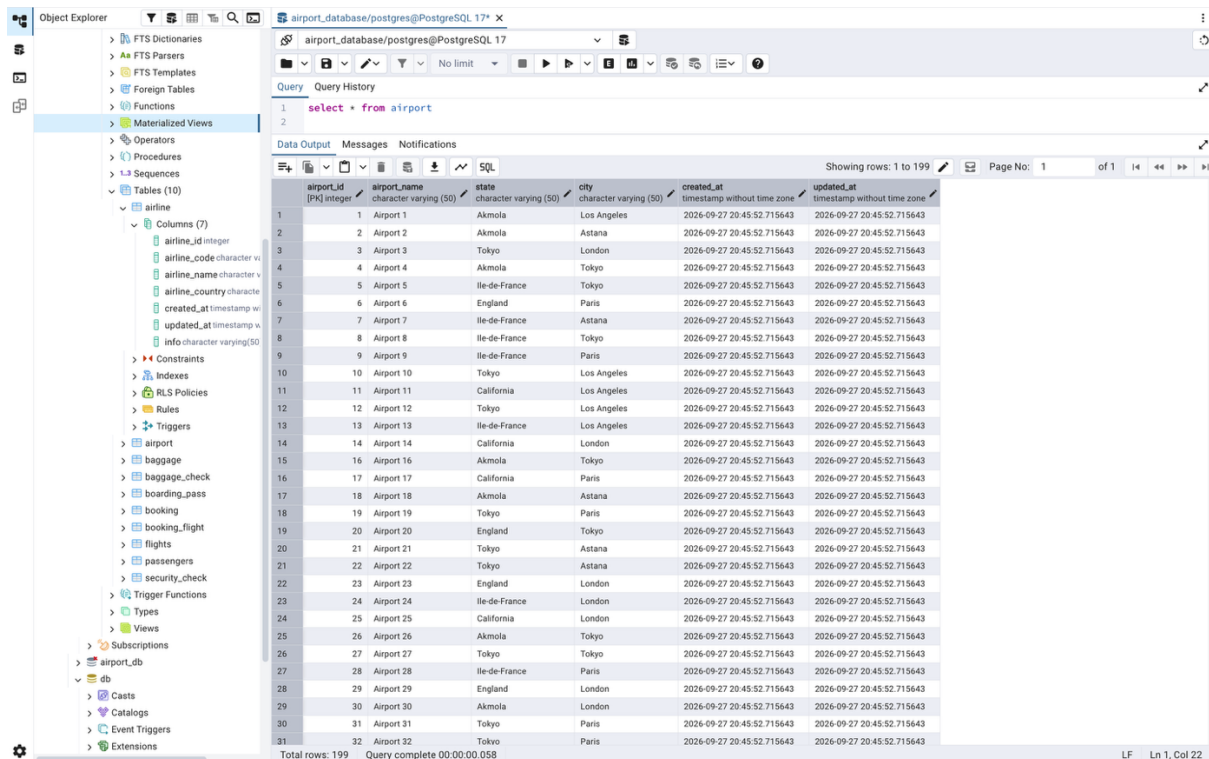
```
1 delete from airport
2 where state is null
3 and city in('Mlawe', 'Kepuh')
4
```

The Data Output pane at the bottom shows the result of the query:

```
DELETE 1
Query returned successfully in 46 msec.
```

A status bar at the bottom right indicates: "Query returned successfully in 46 msec." and "LF Ln 3, Col 22".

Result after deleting:



The screenshot shows the PostgreSQL IDE interface after the deletion. The main query editor contains the following SQL code:

```
1 select * from airport
2
```

The Data Output pane displays the result of the query, showing 199 rows. The columns are: airport_id, airport_name, state, city, created_at, and updated_at. The first few rows are:

airport_id	airport_name	state	city	created_at	updated_at
1	Airport 1	Akmola	Los Angeles	2026-09-27 20:45:52.715643	2026-09-27 20:45:52.715643
2	Airport 2	Akmola	Astana	2026-09-27 20:45:52.715643	2026-09-27 20:45:52.715643
3	Airport 3	Tokyo	London	2026-09-27 20:45:52.715643	2026-09-27 20:45:52.715643
4	Airport 4	Akmola	Tokyo	2026-09-27 20:45:52.715643	2026-09-27 20:45:52.715643
5	Airport 5	Ile-de-France	Tokyo	2026-09-27 20:45:52.715643	2026-09-27 20:45:52.715643

The status bar at the bottom right indicates: "Query complete 00:00:00.058" and "LF Ln 1, Col 22".

Task-11. Insert new unchecked baggage

SQL-Code and Result:

The screenshot shows a PostgreSQL IDE interface. On the left, the Object Explorer displays the database schema, including tables like `airline`, `baggage`, and `baggage_check`. The `baggage_check` table is selected, showing its columns: `baggage_check_id`, `check_result`, `created_at`, `updated_at`, `booking_id`, and `passenger_id`.

The main query editor displays the following SQL code:

```
1 insert into baggage_check(  
2 baggage_check_id,  
3 check_result,  
4 created_at,  
5 updated_at,  
6 booking_id,  
7 passenger_id  
8 )  
9 values(  
10 (select coalesce(max(baggage_check_id), 0) + 1 from baggage_check),  
11 'Not checked',  
12 current_timestamp,  
13 current_timestamp,  
14 1,  
15 1  
16 )  
17 returning baggage_check_id, created_at;  
18
```

The Data Output pane shows the result of the query:

baggage_check_id [PK] integer	created_at timestamp without time zone
1	2026-09-27 22:14:19.742633

The status bar at the bottom indicates: "Total rows: 1 Query complete 00:00:00.060". A green notification bar at the bottom right states: "Successfully run. Total query runtime: 60 msec. 1 rows affected."

Task-12. Convert all country name to uppercasc

SQL-Code:

The screenshot shows the same PostgreSQL IDE interface. The `airline` table is selected in the Object Explorer, showing its columns: `airline_id`, `airline_code`, `airline_name`, `airline_country`, `created_at`, `updated_at`, and `info`.

The main query editor displays the following SQL code:

```
1 update airline  
2 set airline_country = upper(airline_country);  
3
```

The Data Output pane shows the result of the query:

UPDATE 4
Query returned successfully in 98 msec.

The status bar at the bottom indicates: "Total rows: Query complete 00:00:00.098". A green notification bar at the bottom right states: "Query returned successfully in 98 msec."

Result after converting:

The screenshot shows the PostgreSQL IDE interface. On the left, the Object Explorer displays the database schema, including tables like `airline` and `baggage_check`. The main query editor shows the following SQL query:

```
1 select airline_country
2 from airline;
3
```

The Data Output pane displays the results of the query, showing 4 rows of data:

airline_country
TURKEY
FRANCE
BRASIL
POLAND

The status bar at the bottom indicates: "Total rows: 4 Query complete 00:00:00.067". A green message box at the bottom right states: "Successfully run. Total query runtime: 67 msec. 4 rows affected."

Task-13. Update name and country where airline_id = 5

SQL-Code:

The screenshot shows the PostgreSQL IDE interface. The main query editor displays the following SQL query:

```
1 update airline
2 set airline_name = 'Global Airways',
3   airline_country = 'United Kingdom',
4   updated_at = current_date
5 where airline_id = 5;
```

The Data Output pane shows the result of the query:

```
UPDATE 1
Query returned successfully in 68 msec.
```

The status bar at the bottom indicates: "Total rows: Query complete 00:00:00.068". A green message box at the bottom right states: "Query returned successfully in 68 msec."

Result after updating row:

The screenshot shows the PostgreSQL Object Explorer on the left, with the 'airline' table selected under 'Tables (10)'. The 'Columns (7)' list includes: airline_id integer, airline_code character varying(30), airline_name character varying(50), airline_country character varying(50), created_at timestamp without time zone, updated_at timestamp without time zone, and info character varying(50). The main query editor displays the following SQL:

```
1 select * from airline
2 where airline_id = 5;
```

The 'Data Output' tab shows the result of the query:

airline_id	airline_code	airline_name	airline_country	created_at	updated_at	info
5	KR	Global AirWays	United Kingdom	2026-09-27 22:27:29.530475	2026-09-27 00:00:00	[null]

The status bar at the bottom indicates: 'Total rows: 1 Query complete 00:00:00.058' and a green message: 'Successfully run. Total query runtime: 58 msec. 1 rows affected.'

Task-14. Update state name where city ('Astana', 'London', 'Turkey') in airport table

SQL-Code:

The screenshot shows the PostgreSQL Object Explorer on the left, with the 'airport' table selected under 'Tables (10)'. The 'Columns (6)' list includes: baggage_check_id integer, check_result character varying(50), created_at timestamp without time zone, updated_at timestamp without time zone, booking_id integer, and passenger_id integer. The main query editor displays the following SQL:

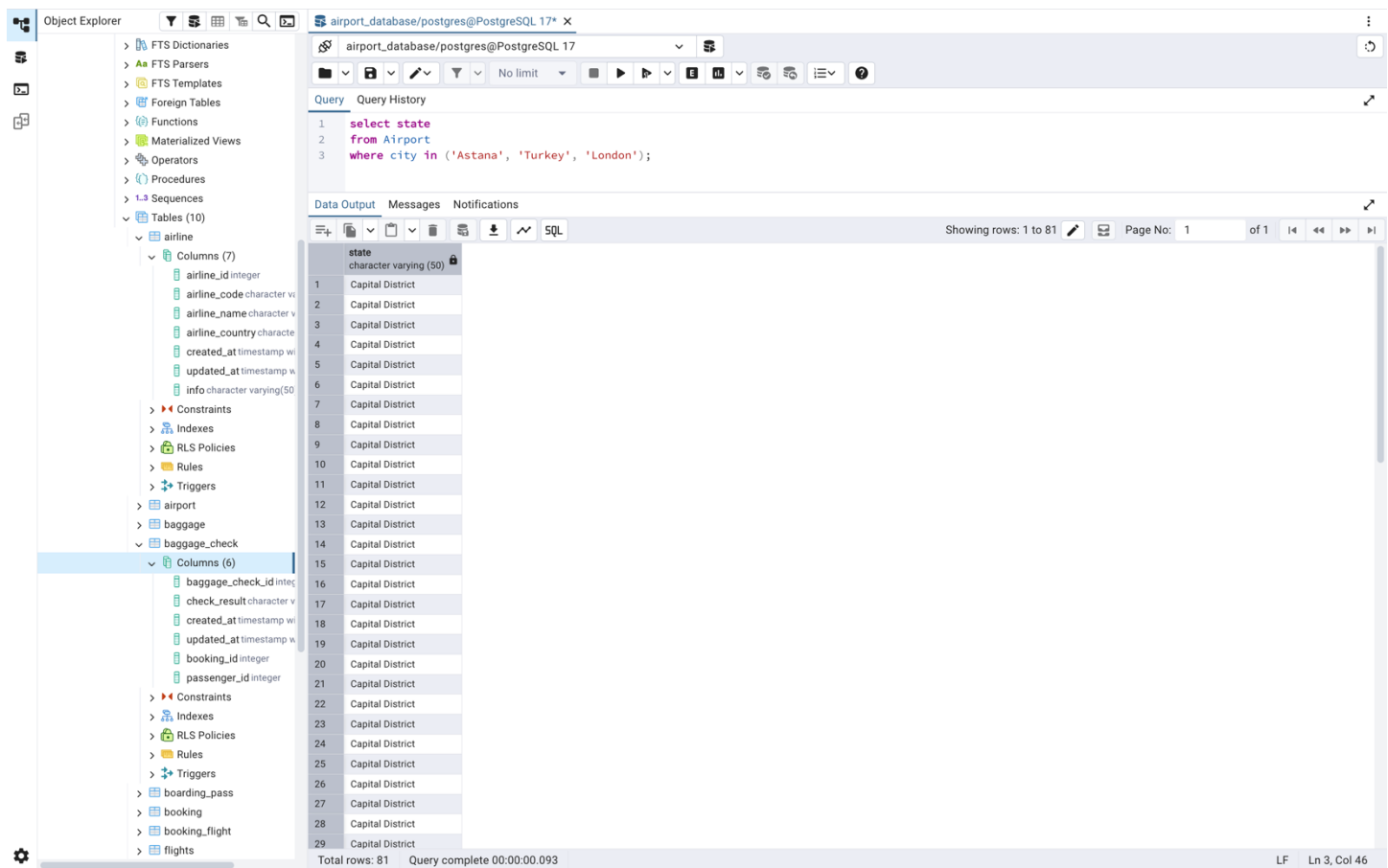
```
1 update airport
2 set state = 'Capital District'
3 where city in ('Astana', 'Turkey', 'London');
```

The 'Data Output' tab shows the result of the query:

UPDATE 81
Query returned successfully in 61 msec.

The status bar at the bottom indicates: 'Total rows: Query complete 00:00:00.061' and a green message: 'Query returned successfully in 61 msec.'

Result after updating:



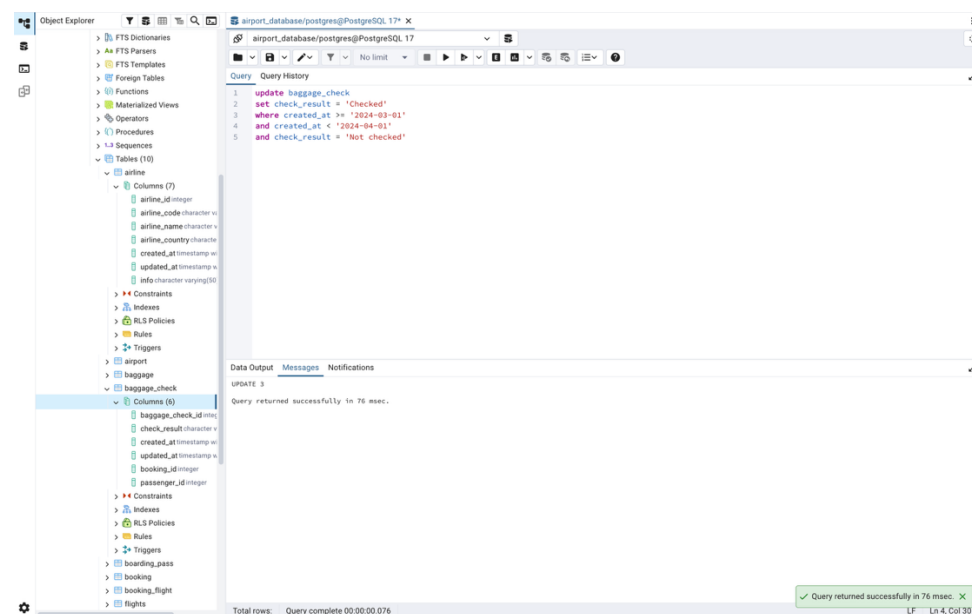
The screenshot displays a PostgreSQL database interface with the following components:

- Object Explorer:** A tree view on the left showing the database structure. The 'airline' table is expanded, showing columns like 'airline_id', 'airline_code', 'airline_name', 'airline_country', 'created_at', 'updated_at', and 'info'. The 'baggage_check' table is also expanded, showing columns like 'baggage_check_id', 'check_result', 'created_at', 'updated_at', 'booking_id', and 'passenger_id'.
- Query Editor:** The central area contains the following SQL query:

```
1 select state
2 from Airport
3 where city in ('Astana', 'Turkey', 'London');
```
- Data Output:** The bottom section shows the results of the query. It displays 29 rows, all with the state 'Capital District'. The status bar indicates 'Total rows: 81' and 'Query complete 00:00:00.093'.

Task-15. Update check_result = “Checked” where created in March 24 and with “Not checked” status.

SQL-Code:



The screenshot displays a PostgreSQL database interface with the following components:

- Object Explorer:** A tree view on the left showing the database structure. The 'baggage_check' table is expanded, showing columns like 'baggage_check_id', 'check_result', 'created_at', 'updated_at', 'booking_id', and 'passenger_id'.
- Query Editor:** The central area contains the following SQL query:

```
1 update baggage_check
2 set check_result = 'Checked'
3 where created_at >= '2024-03-01'
4 and created_at < '2024-04-01'
5 and check_result = 'Not checked'
```
- Data Output:** The bottom section shows the results of the query. It displays 'UPDATE 3' and 'Query returned successfully in 76 msec.' The status bar indicates 'Total rows: Query complete 00:00:00.076'.

Result after updating:

Object Explorer

airline

Columns (7)

airline_id integer

airline_code character v

airline_name character v

airline_country characte

created_at timestamp wi

updated_at timestamp w

info character varying(50

baggage_check

Columns (6)

baggage_check_id inte

check_result character v

created_at timestamp wi

updated_at timestamp w

booking_id integer

passenger_id integer

airline_database/postgres@PostgreSQL 17

Query

select * from baggage_check

Data Output

Showing rows: 1 to 11

Page No: 1 of 1

baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
1	Not checked	2026-09-27 22:14:19.742633	2026-09-27 22:14:19.742633	1	1
2	Checked	2024-03-07 11:20:00	2026-09-27 22:42:41.73226	1	1
3	Checked	2024-03-22 13:10:00	2026-09-27 22:42:41.73226	1	1
4	Not checked	2024-02-15 10:00:00	2026-09-27 22:42:41.73226	1	1
5	Checked	2024-02-25 12:30:00	2026-09-27 22:42:41.73226	1	1
6	Not checked	2024-04-03 09:00:00	2026-09-27 22:42:41.73226	1	1
7	Checked	2024-04-18 15:30:00	2026-09-27 22:42:41.73226	1	1
8	Not checked	2023-12-20 17:00:00	2026-09-27 22:42:41.73226	1	1
9	Checked	2024-03-02 08:30:00	2026-09-27 22:42:41.73226	1	1
10	Checked	2024-03-15 16:45:00	2026-09-27 22:42:41.73226	1	1
11	Checked	2024-03-30 19:00:00	2026-09-27 22:42:41.73226	1	1

Total rows: 11 Query complete 00:00:00.063 LF Ln 1, Col 28